

# claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a7f1c161ae9ace6f4.jsonl`  
- SHA-256: `cf0b4315765f1b090d3de65c53aa927a1c44ad2c60d62f3a6a1716ccfab0f88a`  
- Source modified: `2026-07-02T04:58:31+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_7049e1ed-b93`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

## Transcript

### 1. user (2026-07-02T04:57:37.918Z)

You are an adversarial code reviewer. Today is 2026-07-02. Review PR #419 in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (gh pr view 419, gh pr diff 419; the working copy is in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config ? read code there for context).

The PR claims to fix: R1-18 (CLI --output-dir default clobbers config) (evidence in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT\_REPORT\_khelsutra-guru\_2026-07-02.md ? Grep for the IDs; honor the verifier notes' scoping corrections).

Hunt REAL defects: correctness bugs in the diff, broken/vacuous tests, behavior changes beyond the brief, staging accidents (files that should not be in the diff ? check the file list!), convention violations (default-ON when brief says OFF, missing schema-version check, CWD assumptions), and whether the fix actually addresses the finding. READ-ONLY ? do not modify or comment on GitHub. blocker = must not merge; major = fix before merge; minor = nice-to-have.

### 2. user (2026-07-02T04:57:46.616Z)

```
title: fix(main): --output-dir argparse default no longer clobbers config.json  
output.output_dir  
state: OPEN  
author: avidullu (Avi)  
labels:  
assignees:  
reviewers:  
projects:  
milestone:  
number: 419  
url: https://github.com/Khelsutra/badminton-highlight-indexer/pull/419  
additions: 225  
deletions: 12  
auto-merge: disabled  
--
```

## What

`--output-dir` had argparse default `"output"` and was applied **unconditionally** at `main()` init (`backend/main.py`), so a config-set `output.output\_dir` (the documented knob, `backend/config/models.py::OutputConfig`) could never take effect on the main entry path. The SQLite DB path derives from it, so the server silently created a fresh empty DB under `./output` while `run\_eval.py`/`run\_regression.py` (via `backend.config.default\_db\_path`) resolved the DB from config.json ? split state / perceived data loss. (Audit finding R1-19, 2026-07-02.)

## How

- Argparse default is now `None` (= "not passed").
- New pure helper `backend/utils/paths.py::resolve\_startup\_output\_dir(cli\_output\_dir, config)`: an **explicit** CLI `--output-dir` wins; otherwise config.json's `output.output\_dir` is honored via the existing `output\_base()`; **\*P0a app-home rooting\*** (`resolve\_output\_dir`) applies either way.
- Both `main()` call sites go through the helper: the init path **and** the debug-trim path (which resolved `args.output\_dir` before config load ? per the audit verifier notes it must handle the `None` default; it now falls back to the loaded config value).
- `OutputConfig.output\_dir` comment corrected: the flag overrides only when actually passed.
- **\*\*Byte-identical default\*\***: with neither knob set the result is `output` exactly as before; an explicit `--output-dir` behaves exactly as before.

## Verification

Ran locally (Windows, Python 3.13):

- `python -m pytest tests/test\_startup\_output\_dir.py -q` ? **\*\*12 passed\*\***: pure precedence + `RALLY\_APP\_HOME` rooting units, plus two subprocess wiring tests spawning the real `python -m backend.main` CLI single-shot path and asserting where the DB lands (config-only ? honored, and no clobbered `./output` in the launch CWD; explicit `--output-dir` ? beats a conflicting config value).
- `python -m pytest tests/test\_output\_base.py tests/test\_app\_home.py tests/test\_config.py -q` ? 53 passed.
- `python -m pytest tests/test\_server\_entrypoint.py -q` ? 3 passed (the existing `--output-dir`/P0a/single-instance subprocess entry-point guards).
- `ruff check .` ? clean.
- `mypy backend training` (CI's invocation) ? the only error is `backend/storage/gcs.py:31` (`google.cloud` attr), reproduced **\*\*identically on untouched master\*\*** with the same invocation: a pre-existing local-env artifact, not from this change; the touched modules are clean.

Full suite is CI's job (`run\_all\_tests.py` lane + lint/type gates).

? Generated with [Claude Code](https://claude.com/claude-code)

=====DIFF=====

```
diff --git a/backend/config/models.py b/backend/config/models.py
```

```
index 652693a..980e80c 100644
```

```
--- a/backend/config/models.py
```

```
+++ b/backend/config/models.py
```

```
@@ -272,7 +272,9 @@ class OutputConfig(BaseModel):
```

```
    default_highlights_name: str = "highlights.mp4"
```

```
    db_name: str = "sports_indexer.db"
```

```
    # Directory where the DB, highlight reels, and processed clips are written.
```

```
-    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
```

```
+    # An EXPLICIT CLI --output-dir overrides this at startup ? only when the flag
```

```
+    # is actually passed (backend/utils/paths.py::resolve_startup_output_dir,
```

```
+    # wired in backend/main.py::main).
```

```
    output_dir: str = "output"
```

```
diff --git a/backend/main.py b/backend/main.py
```

```
index 99246fe..6e8377e 100644
```

```
--- a/backend/main.py
```

```
+++ b/backend/main.py
```

```
@@ -22,7 +22,7 @@
```

```
    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) keeps the
    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv().
```

```
    # stdlib-only import ? keeps this pre-pydantic line light.
```

```
-from backend.utils.app_home import env_file_to_load, resolve_output_dir
```

```
+from backend.utils.app_home import env_file_to_load
```

```
    load_dotenv(env_file_to_load())
```

```
@@ -78,6 +78,7 @@
```

```
    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
```

```
    from backend.utils.paths import (
```

```

        output_base,
+       resolve_startup_output_dir,
        resolve_under_root,
        safe_output_filename,
        validate_input_path,
@@ -1836,8 +1837,9 @@ def main():
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument(
        "--output-dir",
-       default="output",
-       help="Directory where processed clips/highlights are saved",
+       default=None, # None = not passed -> honor config.json output.output_dir (default
"output")
+       help="Directory where processed clips/highlights are saved. Overrides "
+       "config.json's output.output_dir only when given.",
    )
    parser.add_argument(
        "--setup",
@@ -1924,9 +1926,10 @@ def main():
        out_dir = os.path.dirname(out_path)
        out_name = os.path.basename(out_path)
    else:
-       out_dir = resolve_output_dir(
-           args.output_dir
-       ) # P0a: app-home aware (CWD-identical when unset)
+       # CLI --output-dir wins when given; otherwise config.json's
+       # output.output_dir (global_config isn't initialized this early).
+       # P0a: app-home aware (CWD-identical when unset).
+       out_dir = resolve_startup_output_dir(args.output_dir, load_app_config())
        out_path = os.path.join(out_dir, out_filename)
        out_name = out_filename

@@ -1984,12 +1987,16 @@ def main():
    global_config
    ) # M5: fail fast on an incoherent ACTIVE profile

-   # The CLI --output-dir overrides the configured output directory. It now lives
-   # on the typed model (OutputConfig.output_dir), so every consumer ? including
-   # /api/compile ? reads the same value instead of a write-only runtime hack.
+   # An EXPLICIT CLI --output-dir overrides the configured output directory; without
+   # the flag, config.json's output.output_dir is honored (the argparse default used
+   # to clobber it on every start). It lives on the typed model (OutputConfig.output_dir),
+   # so every consumer ? including /api/compile ? reads the same value instead of a
+   # write-only runtime hack.
    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
-   # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
-   global_config.output.output_dir = resolve_output_dir(args.output_dir)
+   # unset and neither knob set, the result is "output" (CWD-relative) ? byte-identical.
+   global_config.output.output_dir = resolve_startup_output_dir(
+       args.output_dir, global_config
+   )
    os.makedirs(global_config.output.output_dir, exist_ok=True)

    db_path = os.path.join(
diff --git a/backend/utils/paths.py b/backend/utils/paths.py
index 1ed2e7d..979df74 100644
--- a/backend/utils/paths.py
+++ b/backend/utils/paths.py
@@ -3,6 +3,8 @@
import os
from typing import Any, Optional

+from backend.utils.app_home import resolve_output_dir
+

```

```

def output_base(config: Any) -> str:
    """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
@@ -24,6 +26,26 @@ def output_base(config: Any) -> str:
    return out or "output"

+def resolve_startup_output_dir(cli_output_dir: Optional[str], config: Any) -> str:
+    """The EFFECTIVE output dir at process startup (``backend.main::main``).
+
+    Precedence: an EXPLICIT CLI ``--output-dir`` wins; without it (``None``),
+    config.json's ``output.output_dir`` is honored via :func:`output_base`
+    (default ``"output"`` ? byte-identical to the old always-applied argparse
+    default). Either way P0a app-home rooting applies
+    (:func:`backend.utils.app_home.resolve_output_dir`: a RELATIVE dir roots
+    under ``RALLY_APP_HOME`` when set; an absolute one passes through).
+
+    Regression guard: the argparse default used to be ``"output"`` and was
+    applied UNCONDITIONALLY, silently clobbering the documented
+    ``output.output_dir`` knob (backend/config/models.py::OutputConfig) ? and
+    moving the SQLite DB with it ? on every server/CLI start.
+    """
+    if cli_output_dir is not None:
+        return resolve_output_dir(cli_output_dir)
+    return resolve_output_dir(output_base(config))
+
def safe_output_filename(name: str) -> str:
    """Validate an output filename is a BARE name (no dir part, no traversal, not absolute).

diff --git a/tests/test_startup_output_dir.py b/tests/test_startup_output_dir.py
new file mode 100644
index 00000000..8787081
--- /dev/null
+++ b/tests/test_startup_output_dir.py
@@ -0,0 +1,182 @@
+"""Startup output-dir precedence: CLI --output-dir > config.json output.output_dir > "output".
+
+Regression guard for the audit finding where the argparse default ``"output"`` was applied
+UNCONDITIONALLY at ``backend.main::main`` init, silently clobbering the documented
+``output.output_dir`` knob (backend/config/models.py::OutputConfig) ? and moving the SQLite DB
+with it ? on every server/CLI start. The fix: argparse default is ``None`` and
+``backend.utils.paths.resolve_startup_output_dir`` only lets the CLI win when the flag was
+actually passed.
+
+Two layers:
+* pure unit tests of ``resolve_startup_output_dir`` (precedence + P0a app-home rooting);
+* subprocess wiring tests spawning the real CLI single-shot path (no server, no port ?
+the dummy video fails fast AFTER the DB is created) asserting where the DB lands.
+"""
+
+import json
+import os
+import subprocess
+import sys
+
+import pytest
+
+from backend.config.models import AppConfig
+from backend.utils import app_home as ah
+from backend.utils.paths import resolve_startup_output_dir
+
+REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
+
+@pytest.fixture(autouse=True)

```

```

+def _clear_app_home_env(monkeypatch):
+    """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
+    monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
+
+
+## ----- unit: precedence
+
+
+def test_neither_set_is_byte_identical_default():
+    # No CLI flag, all-defaults config ? the historical literal "output".
+    assert resolve_startup_output_dir(None, AppConfig()) == "output"
+
+
+def test_config_only_is_honored():
+    """The exact audit bug: a config-set output_dir must survive startup without the flag."""
+    cfg = AppConfig()
+    cfg.output.output_dir = "D:/rally-data"
+    assert resolve_startup_output_dir(None, cfg) == "D:/rally-data"
+
+
+def test_config_only_relative_is_verbatim_when_env_unset():
+    cfg = AppConfig()
+    cfg.output.output_dir = "reels"
+    assert resolve_startup_output_dir(None, cfg) == "reels"
+
+
+def test_cli_wins_over_config():
+    cfg = AppConfig()
+    cfg.output.output_dir = "D:/rally-data"
+    assert resolve_startup_output_dir("C:/cli-dir", cfg) == "C:/cli-dir"
+
+
+def test_empty_config_value_falls_back_to_output():
+    cfg = AppConfig()
+    cfg.output.output_dir = ""
+    assert resolve_startup_output_dir(None, cfg) == "output"
+
+
+def test_empty_cli_string_falls_back_to_output():
+    # `--output-dir ""` is "passed" but empty ? the historical default, not a crash.
+    cfg = AppConfig()
+    cfg.output.output_dir = "D:/rally-data" # CLI (even empty) still wins over config
+    assert resolve_startup_output_dir("", cfg) == "output"
+
+
+def test_legacy_dict_shim_config_works():
+    assert (
+        resolve_startup_output_dir(None, {"output": {"output_dir": "shim_out"}})
+        == "shim_out"
+    )
+
+
+## ----- unit: P0a app-home rooting
+
+
+def test_config_relative_roots_under_app_home(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
+    cfg = AppConfig()
+    cfg.output.output_dir = "reels"
+    assert resolve_startup_output_dir(None, cfg) == str(tmp_path / "reels")
+
+
+def test_cli_relative_roots_under_app_home(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
+    assert resolve_startup_output_dir("custom", AppConfig()) == str(

```

```

+         tmp_path / "custom"
+     )
+
+
+def test_config_absolute_passes_through_app_home(monkeypatch, tmp_path):
+    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
+    absolute = str(tmp_path / "elsewhere")
+    cfg = AppConfig()
+    cfg.output.output_dir = absolute
+    assert resolve_startup_output_dir(None, cfg) == absolute
+
+
+## ----- wiring: the real main() init
+
+
+def _run_single_shot(launch_dir, extra_args=()):
+    """Spawn the CLI single-shot path (no server): inits config + DB, fails fast on the
+    dummy video, exits. Mirrors tests/test_server_entrypoint.py's P0a wiring test."""
+    video = launch_dir / "dummy.mp4"
+    video.write_bytes(b"not-a-real-video") # exists; motion fails fast after DB init
+    env = {
+        *os.environ,
+        "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),
+    }
+    env.pop(ah.APP_HOME_ENV, None) # CWD-relative resolution, like a bare dev run
+    return subprocess.run(
+        [
+            sys.executable,
+            "-m",
+            "backend.main",
+            "--video-path",
+            str(video),
+            "--segmenter",
+            "motion",
+            *extra_args,
+        ],
+        cwd=str(launch_dir),
+        env=env,
+        capture_output=True,
+        text=True,
+        timeout=180,
+    )
+
+
+def test_main_honors_config_output_dir_without_cli_flag(tmp_path):
+    """End-to-end proof of the fix: output.output_dir set ONLY in config.json moves the DB
+    there ? and no clobbered ./output is created in the launch CWD."""
+    db_name = AppConfig().output.db_name
+    launch = tmp_path / "launch"
+    cfg_dir = tmp_path / "from_config"
+    launch.mkdir()
+    (launch / "config.json").write_text(
+        json.dumps({"output": {"output_dir": str(cfg_dir)}}), encoding="utf-8"
+    )
+
+    proc = _run_single_shot(launch)
+
+    assert (cfg_dir / db_name).exists(), (
+        "DB was NOT created under config.json's output.output_dir ? the CLI default "
+        "clobbered the configured knob again.\n"
+        f"stdout:\n{proc.stdout[-1500:]}\\nstderr:\n{proc.stderr[-1500:]}"
+    )
+
+    assert not (launch / "output").exists(), (
+        "output/ appeared in the launch CWD ? the argparse default overrode the config"
+    )

```

```

+
+
+def test_main_cli_output_dir_beats_config(tmp_path):
+    """An explicit --output-dir still wins over a conflicting config.json value."""
+    db_name = AppConfig().output.db_name
+    launch = tmp_path / "launch"
+    cfg_dir = tmp_path / "from_config"
+    cli_dir = tmp_path / "from_cli"
+    launch.mkdir()
+    (launch / "config.json").write_text(
+        json.dumps({"output": {"output_dir": str(cfg_dir)}}), encoding="utf-8"
+    )
+
+    proc = _run_single_shot(launch, extra_args=("--output-dir", str(cli_dir)))
+
+    assert (cli_dir / db_name).exists(), (
+        "DB was NOT created under the explicit --output-dir.\n"
+        f"stdout:\n{proc.stdout[-1500:]}\\nstderr:\n{proc.stderr[-1500:]}\"
+    )
+    assert not (cfg_dir / db_name).exists(), (
+        "DB leaked into config.json's output_dir despite an explicit --output-dir"
+    )

```

### 3. user (2026-07-02T04:57:46.749Z)

190-- evidence: §7 tracker (lines 63-73): P1-P8 all ?. Verified in code: grep 'build\_stitch\_plan' across backend/ and tests/ returns nothing (P2's pure stitch-plan seam was never factored out of backend/pipeline/stitcher/compiler.py), and tests/test\_golden\_fixtures.py has no stitch-plan snapshot. Minor rot: the P0 row (line 65) still shows ? '(this PR)' though the doc merged as #383 (commit 1283e18, 2026-06-25) and the header still says 'DRAFT'.

191-- why it matters: This gate exists precisely to bless genuine (non-AST-identical) refactors as quality-neutral ? i.e., exactly the class of change as the stalled #390 main.py split. Executing P1-P4 first would de-risk redoing the god-module split; leaving it a week-stale locked-decisions doc means refactors continue with no behavioral proof.

192-- proposed fix: Start §7 P1+P2 (canonical decision-snapshot schema + factor pure build\_stitch\_plan() out of compiler.py) as the next Claude-doable engine work; flip the P0 row to ? #383 and status to IN PROGRESS in the same PR. Ideally land P1-P4 before re-cutting the main.py split.

193-[Omitted long context line]

194-[Omitted long context line]

195-

196-### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer

197-\*\*Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud\_run marker reads) ? 'OS temp reclamation' premise is false on Windows\*\*

198-- anchor: `backend/storage/\_\_init\_\_.py:89` · tracked in: ? · finder: engine:correctness

199-[Omitted long context line]

200-- why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.

201-- proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output\_base(cfg)/tmp with an age-based sweep at startup; make cloud\_run.\_read\_json use TemporaryDirectory.

202-[Omitted long context line]

203-[Omitted long context line]

204-

205-### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer

206-\*\*GDriveApiStorage.fetch\_to\_local buffers the entire file in RAM (get\_media().execute()); put() deletes the old object before the new upload succeeds\*\*

207-- anchor: `backend/storage/gdrive\_api.py:144` · tracked in: ? · finder: engine:correctness

208-- evidence: gdrive\_api.py:143-148: `data = self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)` ? the whole object is materialized as one bytes value (no MediaIoBaseDownload streaming). gdrive\_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)` BEFORE `files().create(...)` ? delete-then-create is not atomic.

209-- why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes

live for the reel round-trip.

210-- proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for put(), create the new file first (or upload to a temp name) and delete the old one only after success.

211-[Omitted long context line]

212-[Omitted long context line]

213-

214-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer

215-**\*\*config.json output.output\_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it\*\***

216-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness

217-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db\_path derives from it (main.py:1995-1997).

218-- why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).

219-- proposed fix: Change argparse default to None and only override when args.output\_dir is not None (falling back to the config value through resolve\_output\_dir).

220-[Omitted long context line]

#### 4. user (2026-07-02T04:57:57.528Z)

196     ### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer

197     **\*\*Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud\_run marker reads) ? 'OS temp reclamation' premise is false on Windows\*\***

198     - anchor: `backend/storage/\_\_init\_\_.py:89` · tracked in: ? · finder: engine:correctness

199     - evidence: storage/\_\_init\_\_.py:80-90: acquire\_local\_input docstring says the scratch copy 'is left under the system temp dir ... and relies on OS temp reclamation'. Call sites that leak one full-video copy per run: \_execute\_processing (main.py:1168-1171), \_execute\_compile (main.py:1683-1686), create\_asset (main.py:791-799), get\_source (main.py:646).

cloud\_run.py:219-222 additionally leaks a cr-read-\* mkdtemp per marker/report read. No janitor exists (grep for rally-input-/cleanup found none); engine issue #301 covers only the upload\_dir retention sweep, not temp scratch.

200     - why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.

201     - proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output\_base(cfg)/tmp with an age-based sweep at startup; make cloud\_run.\_read\_json use TemporaryDirectory.

202     - verifier notes: Two sharpenings. (a) get\_source (main.py:646) cannot use a simple finally-delete: the FileResponse streams from the scratch file after the handler returns, so deletion there needs the age-based-sweep option (or a keyed scratch cache reused across views) ? the finder's second proposed option covers it; per-job finally scopes fit \_execute\_processing/\_execute\_compile/create\_asset. (b) Add a fifth leaking call site to the fix scope: the CLI single-shot path at main.py:2056. Sharper failure scenario: with input\_backend=gdrive-api/gcs, each processed match leaks one full multi-GB copy into %TEMP% per process job, another per compile, one per /api/assets source\_uri registration (fetched solely to hash), and one per /api/videos/{id}/source view ? until require\_free\_bytes(tempfile.gettempdir()) at main.py:789/1402 starts rejecting all further remote ingests at stage="fetch" even though the disk is full of the tool's own dead scratch.

203     - verifier reasoning: Personally verified in the current tree. (1) backend/storage/\_\_init\_\_.py:80-82 docstring says the scratch copy "is left under the system temp dir ? and relies on OS temp reclamation"; line 89 does tempfile.mkdtemp(prefix="rally-input-") and line 90 fetches into it ? no deletion. (2) Every claimed call site leaks: \_execute\_processing main.py:1168-1171 (no rmtree anywhere in main.py ? grep confirmed rmtree exists only in eval/golden\_real\_fixtures.py, pipeline/stitcher/compiler.py, pipeline/detectors/\*); \_execute\_compile main.py:1683-1686; create\_asset main.py:791-799 fetches a full video copy just to MD5-hash it (main.py:805) then returns at 865-875 without deleting local\_working; get\_source main.py:646 fetches per-request and serves it via FileResponse (line 660) ? a per-view full-video leak for remote sources. Plus a fifth site the finder missed: CLI single-shot main.py:2056. cloud\_run.py:219-222 \_read\_json mkdtemps "cr-read-" per marker/report read, no cleanup. (3) No janitor: tools/cleanup\_caches.py scans only WSL ~/clips, the output/



tree, and security.upload\_dir (docstring lines 3-5, classify() lines 52-97) ? never  
tempfile.gettempdir(); its "tmp\*" rule (line 79-80) applies only inside those trees. (4) Not  
fixed: git log for storage/\_\_init\_\_.py tops out at b6214e5 (repo-wide ruff format); gh issue 301  
confirmed to cover only the T11 upload-retention scheduled job, not temp scratch. (5) Guard  
interplay real: require\_free\_bytes(tempfile.gettempdir(), ?, stage="fetch") at main.py:789 and  
main.py:1402 ? leaked copies shrink exactly the headroom the fetch guard checks, so accumulation  
converts into false fetch rejections. Windows does not auto-reclaim %TEMP% by default, so the  
docstring premise is false on the shipping platform. Severity P2 honest: affects only remote-  
input flows (local paths are identity passthrough, \_\_init\_\_.py:85-88), slow-burn, no data loss.  
Not owner-gated ? pure disk hygiene, no strategy/spend/ToS/corpus decision.

204

205 **### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer**

206 **\*\*GDriveApiStorage.fetch\_to\_local buffers the entire file in RAM**

(get\_media().execute()); put() deletes the old object before the new upload succeeds\*\*

207 - anchor: `backend/storage/gdrive\_api.py:144` · tracked in: ? · finder:

engine:correctness

208 - evidence: gdrive\_api.py:143-148: `data =

self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)` ? the whole  
object is materialized as one bytes value (no MediaIoBaseDownload streaming).

gdrive\_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`  
BEFORE `files().create(...)` ? delete-then-create is not atomic.

209 - why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it  
goes live for the reel round-trip.

210 - proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in  
chunks; for put(), create the new file first (or upload to a temp name) and delete the old one  
only after success.

211 - verifier notes: File/line accurate (line 144 is the get\_media().execute() call).

Failure scenarios: (1) fetch\_to\_local on a multi-GB source/reel buffers the full file in RAM on  
a memory-limited Cloud Run box ? OOM kill; (2) re-put() of a reel to an existing path deletes  
the old file, then the resumable create fails (network/quota) ? the reel is gone with no copy at  
either end. Fix refinement: for put(), prefer `files().update(fileId=existing, media\_body=...)`  
when a file already exists ? atomic in-place content replacement that preserves the file id and  
avoids both the delete-first data-loss window and the duplicate-(name,parent) window that  
create-first-then-delete would open (the file's own comment at lines 160-162 notes \_resolve\_id  
returns the first match among duplicates). Download fix as proposed: MediaIoBaseDownload chunked  
streaming.

212 - verifier reasoning: backend/storage/gdrive\_api.py:143-148 ? fetch\_to\_local does `data  
= self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)`: the whole  
object is returned as a single bytes value; no MediaIoBaseDownload/chunked loop exists in the  
file (only MediaFileUpload for uploads, lines 78-84). Docstring (lines 3-6, 19) states this  
backend serves videos/reels on headless Cloud Run/GPU hosts, so multi-GB buffering is a genuine  
OOM risk. backend/storage/gdrive\_api.py:163-170 ? put() resolves `existing`, calls  
`files().delete(fileId=existing).execute()`, and only then `files().create(...)`: a create  
failure after the delete loses the prior object (non-atomic overwrite). git log --  
backend/storage/gdrive\_api.py shows only formatting/isort commits (b6214e5, f523d3a) since the  
introducing commit 819e395 (M12, PR #338) ? not fixed. Not owner-gated: the module's owner gate  
(lines 14-17) covers the per-user OAuth flow, not this code's download/upload mechanics.

213

214 **### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer**

215 **\*\*config.json output.output\_dir is silently clobbered by the CLI --output-dir DEFAULT on  
every server/CLI start ? the typed knob is dead, and the DB path moves with it\*\***

216 - anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness

217 - evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992  
unconditionally does `global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir)` ?  
the override fires even when the flag was never passed, so a config.json value can never take  
effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir  
overrides this at startup' ? implying only when given). db\_path derives from it  
(main.py:1995-1997).

218 - why it matters: A documented config knob that is a no-op on the main entry path, with  
data-location consequences (perceived data loss / split state).

219 - proposed fix: Change argparse default to None and only override when args.output\_dir  
is not None (falling back to the config value through resolve\_output\_dir).

220 - verifier notes: Path correction: the documented knob is at

backend/config/models.py:274-276, not backend/models.py (which does not exist). Sharper failure  
scenario: user sets output.output\_dir: "D:/rally-data" in config.json and restarts `indexer`

--app`; main.py:1992 resets the dir to resolve\_output\_dir("output"), a fresh empty DB is created there (main.py:1995-1997) so the UI shows zero videos (perceived data loss), while run\_eval.py/run\_regression.py via backend/config/\_\_init\_\_.py:41-42 resolve the DB at D:/rally-data ? server and eval tooling disagree on DB location. Fix scope addition: the trim path at backend/main.py:1927-1929 also calls resolve\_output\_dir(args.output\_dir) before config load and must handle a None default (fall back to the loaded config value there too, or to "output" since global\_config isn't initialized yet at that point). Effort stays S.

221 - verifier reasoning: Read current tree. backend/main.py:1837-1841 defines --output-dir with default="output". backend/main.py:1982 loads the typed config; backend/main.py:1992 then unconditionally does global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir), which fires even when the flag was never passed. resolve\_output\_dir (backend/utils/app\_home.py:83-100) never consults the config value ? with RALLY\_APP\_HOME unset it returns the argparse default "output" verbatim ? so a config.json output.output\_dir can never take effect. db\_path derives from the clobbered value (backend/main.py:1995-1997). The only production entry is main() ? create\_app (main.py:2184) ? uvicorn.run (main.py:2185); there is no module-level app, so every server/CLI start hits the clobber. The knob is documented at backend/config/models.py:274-276. Not fixed: git log --oneline -15 -- backend/main.py tops out at e410136 (#413), none touching this. Split-state is concrete: backend/config/\_\_init\_\_.py:41-42 (default\_db\_path, used by run\_eval.py/run\_regression.py) builds the DB path directly from config.json's output.output\_dir with no clobber ? so with the knob set, the server and the eval tooling resolve DIFFERENT DB locations. Not owner-gated (local config/CLI startup only).

222

223 ### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer

224 \*\*upload\_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live under RALLY\_APP\_HOME\*\*

225 - anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness

226 - evidence: uploads.py:50 `upload\_dir = getattr(sec, 'upload\_dir', None) or 'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no resolve\_output\_dir/app\_path is ever applied to it (only output.output\_dir is resolved, main.py:1992 via app\_home.py:83-100). models.py:336 default 'output/uploads'.

227 - why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly the class of CWD-vs-app-home bug P0a was built to eliminate (app\_home.py:1-8).

228 - proposed fix: Resolve upload\_dir through resolve\_output\_dir()/app\_path() in \_upload\_cfg (relative ? app-home-rooted; absolute passes through), mirroring output\_dir.

229 - verifier notes: Sharper failure scenario: on macOS the .command runs with Terminal's CWD=\$HOME, and any `indexer --app` shell launch (advertised in install.py:243) uses the caller's CWD ? uploads land in e.g. ~/output/uploads while DB/reels live under ~/Library/Application Support/Khelsutra/output; unwritable CWD ? PermissionError at uploads.py:89 ? 500 on /api/upload/init. Caveat: Windows double-click of KhelSutra.bat coincidentally works (the .bat lives in app\_home, install.py:175, and cmd sets CWD to the .bat dir). Extra impact the finder missed: tools/cleanup\_caches.py:403 sweeps <output-dir>/uploads (app-home-rooted post-P0a) and the uninstall manifest user\_data (install.py:117-121) lists only app-home paths, so stray multi-GB uploads are never swept and survive uninstall --purge. Fix as proposed is sane: resolve\_output\_dir() in \_upload\_cfg is byte-identical with RALLY\_APP\_HOME unset (app\_home.py:94-95) and existing tests use absolute tmp paths (test\_upload\_download.py:31) that pass through.

230 - verifier reasoning: uploads.py:50 `upload\_dir = getattr(sec, "upload\_dir", None) or "output/uploads"` used verbatim at :89 os.makedirs, :93, :95, :160, :170 ? no resolver applied (repo-wide grep). main.py:1992 resolves ONLY output.output\_dir via resolve\_output\_dir (app\_home.py:83-100); models.py:336 default is relative "output/uploads"; the packaged launcher (install.py:81-99) sets RALLY\_APP\_HOME but never cd's, and the first-run seeded config (loader.py:261-265 LIGHT\_DEFAULT\_CONFIG) has no security block, so the relative default is live in every packaged install. git log -- backend/api/uploads.py (top f28af73) shows no fix. Not owner-gated (local packaging correctness).

231

232 ### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer

233 \*\*Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no cancellation, so the L4 keeps billing and its late result is never ingested\*\*

234 - anchor: `backend/compute/cloud\_run.py:173` · tracked in: ? · finder: engine:correctness

235 - evidence: cloud\_run.py:173-178: run\_infer blocks on poll\_until(...); execution\_policy.py:183-189 raises PolicyTimeout after max\_wait\_sec (default 1800s). The exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ? no code anywhere in cloud\_run.py calls executions.cancel/delete (grep: no cancel API), and the

late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).

236 - why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-gated go-live area, but the code path is live behind compute='cloud').

237 - proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API (executions.cancel with the stored execution name) and log the execution id in the failure message; optionally check the done-marker once more before failing.

238 - verifier notes: Correction to the failure scenario: "the L4 keeps billing" is bounded, not open-ended ? the job is created with --max-retries 0 --task-timeout 3600 (deploy/cloudrun/README.md:33), so the orphaned execution bills at most ~30 extra minutes past the 1800s poll deadline, then Cloud Run kills it. The sharper, systematic scenario is the policy/task-timeout mismatch (1800s poll vs 3600s task budget): a long match video whose GPU inference takes 30-60 min ALWAYS ends as a local "cloud dispatch error" failure while the cloud run later succeeds, writes the done-marker + outputs to the bucket, and the result is silently wasted ? user pays full GPU cost and must re-dispatch. Fix scoping corrections: (a) cancellation must be a new abstract method on CloudRunJobClient (the injected test seam) with the real impl using run\_v2.ExecutionsClient().cancel\_execution, owner-verified like run\_job since the SDK is not a CI dep; (b) it must be best-effort ? run\_job's return falls back to the literal string "execution" when op.metadata.name is absent (cloud\_run.py:108-112), which is not a resolvable execution name; (c) the "check the marker once more before failing" suggestion is good and cheap; also consider aligning max\_wait\_sec with the job's task-timeout. Owner-gating: the shim fix + fake-client tests are doable now; only the real-client cancel verification and any policy retune on the live job are owner-gated (M7 real-run / real cloud spend convention), which matches the finding's own caveat.

239 - verifier reasoning: Personally reproduced from current code (post-pull, HEAD e410136). (1) backend/compute/cloud\_run.py:173-178 ? run\_infer blocks on poll\_until(self.\_read\_marker(done\_uri), self.\_policy) with no exception handling; backend/infrastructure/execution\_policy.py:54 sets max\_wait\_sec=1800.0 default and :183-189 raises PolicyTimeout at the deadline. (2) backend/main.py:1064-1070 catches the exception and returns \_failed(f"cloud dispatch error: {e}", True) ? the local job goes terminal-failed, so the late done-marker/outputs are never ingested. (3) No cancellation exists anywhere: grep for cancel/executions./CancelExecution across backend/\*.py hits only the minified UI asset (backend/ui/assets/index-ukuS9wxa.js); GoogleCloudRunJobClient (cloud\_run.py:69-112) only implements run\_job, and the CloudRunJobClient ABC (:50-66) has no cancel method. (4) Not already fixed: git log -15 -- backend/compute/cloud\_run.py shows only formatting (b6214e5), isort (f523d3a), an unrelated project-guard fix (688d28f), and the original M6/provenance commits. (5) The mechanism is worse than a rare edge: get\_compute\_backend (backend/compute/\_\_init\_\_.py:39,76) passes DEFAULT\_POLICY (1800s) while the deployed job uses --task-timeout 3600 (deploy/cloudrun/README.md:33, docs/CONVERT\_GCS\_VIDEOS\_TO\_HIGHLIGHTS.md:57), so any execution legitimately taking 30-60 min is systematically abandoned at 30 min, keeps billing the L4 for up to another 30 min, then its completed result is dropped. deploy/cloudrun/README.md:60-64 documents the timeout path but only for hung/crashed containers ? the slow-but-healthy case was not considered.

240

## 5. user (2026-07-02T04:58:00.683Z)

CLAUDE.md  
 README.md  
 backend  
 config.json  
 deploy  
 docs  
 eval\_baselines  
 mypy.ini  
 pyproject.toml  
 pytest.ini  
 requirements.txt  
 ruff.toml  
 run\_all\_tests.py  
 run\_eval.py  
 run\_regression.py  
 scratch  
 scripts  
 tests  
 tools

```

training
8db9175 fix(main): stop the --output-dir argparse default from clobbering config.json
output.output_dir
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
  backend/config/models.py          |   4 +-
  backend/main.py                   |  29 ++++---
  backend/utils/paths.py            |  22 ++++
  tests/test_startup_output_dir.py | 182 +++++
4 files changed, 225 insertions(+), 12 deletions(-)

```

## 6. user (2026-07-02T04:58:30.133Z)

```

1      """Resolved application-data HOME for a packaged, batteries-included install.
2
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
6      from
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
8      silently
9      "loses" their index. This module introduces ONE resolver every state path roots on.
10
11     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
12     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
13     sets it).
14     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
15     behaviour.
16
17     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
18     ``~/local/share``)
19     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
20     launcher (P2)
21     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
22     importing
23     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
24     writes into a
25     real user profile during a test run).
26
27     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
28     before
29     the heavier pydantic-backed ``backend.config`` package ? keeping the early
30     ``load_dotenv`` path light.
31
32     """
33
34     from __future__ import annotations
35
36     import os
37     import platform
38     from pathlib import Path
39
40     #: Environment variable the packaged launcher sets to pin the app-data home.
41     APP_HOME_ENV = "RALLY_APP_HOME"
42
43     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
44     APP_DIR_NAME = "Khelsutra"
45
46     def app_home() -> Path:
47         """The resolved application-data home.
48
49         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
50         byte-identical

```

```

40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """`<app-home>/config.json` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """`<app-home>/env` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63         return app_home() / ".env"
64
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if (
76             str(home) == "."
77         ): # env unset (or pinned to CWD) ? historical load_dotenv() search
78             return None
79         candidate = home / ".env"
80         return candidate if candidate.exists() else None
81
82
83     def resolve_output_dir(output_dir: str) -> str:
84         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
85
86         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
87         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
88         normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
89         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
90         app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes

```

```

91     through untouched.
92     """
93     home = app_home()
94     if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
95         return output_dir or "output"
96     if not output_dir:
97         return str(home / "output")
98     if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
99         return output_dir
100     return str(home / output_dir)
101
102
103 def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
104     """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
105     ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
106
107     * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
108     * macOS   ? ``~/Library/Application Support/<app_name>``
109     * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
110     """
111     system = platform.system()
112     if system == "Windows":
113         base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
114     elif system == "Darwin":
115         base = str(Path.home() / "Library" / "Application Support")
116     else:
117         base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
118     return Path(base) / app_name
119

```

## 7. user (2026-07-02T04:58:30.848Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3     import os
4     from typing import Any, Optional
5
6     from backend.utils.app_home import resolve_output_dir
7
8
9     def output_base(config: Any) -> str:
10         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
11         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
12         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
13         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
14
15         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
16         ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy
dict-shim
17         (``.get("output", {})).get("output_dir")``); any miss or empty value falls back to
``"output"``. """
18         try:
19             out = (
20                 config.output.output_dir
21                 if not isinstance(config, dict)
22                 else config.get("output", {}).get("output_dir")
23             )
24         except Exception:
25             out = None
26         return out or "output"

```

```

27
28
29 def resolve_startup_output_dir(cli_output_dir: Optional[str], config: Any) -> str:
30     """The EFFECTIVE output dir at process startup (`backend.main::main`).
31
32     Precedence: an EXPLICIT CLI `--output-dir` wins; without it (`None`),
33     config.json's `output.output_dir` is honored via :func:`output_base`
34     (default `output` ? byte-identical to the old always-applied argparse
35     default). Either way P0a app-home rooting applies
36     (:func:`backend.utils.app_home.resolve_output_dir`: a RELATIVE dir roots
37     under `RALLY_APP_HOME` when set; an absolute one passes through).
38
39     Regression guard: the argparse default used to be `output` and was
40     applied UNCONDITIONALLY, silently clobbering the documented
41     `output.output_dir` knob (backend/config/models.py::OutputConfig) ? and
42     moving the SQLite DB with it ? on every server/CLI start.
43     """
44     if cli_output_dir is not None:
45         return resolve_output_dir(cli_output_dir)
46     return resolve_output_dir(output_base(config))
47
48
49 def safe_output_filename(name: str) -> str:
50     """Validate an output filename is a BARE name (no dir part, no traversal, not
51     absolute).
52     `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
53     absolute,
54     and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
55     write
56     primitive. Returns the name unchanged when safe; raises ValueError otherwise.
57     """
58     if not name or not name.strip():
59         raise ValueError("output_filename must not be empty")
60     if "\x00" in name:
61         raise ValueError("output_filename must not contain a null byte")
62     if "/" in name or "\\" in name:
63         raise ValueError(f"output_filename must not contain a path separator: {name!r}")
64     if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
65         raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
66     return name
67
68 def resolve_under_root(path: str, root: str) -> str:
69     """Return the real path, asserting it is contained under `root` (raises ValueError
70     if not)."""
71     rp = os.path.realpath(path)
72     rr = os.path.realpath(root)
73     try:
74         contained = os.path.commonpath([rp, rr]) == rr
75     except ValueError: # different drives on Windows
76         contained = False
77     if not contained:
78         raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
79     return rp
80
81 def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
82     """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
83
84     With no allowed_root (default, single-user local mode) only the null-byte check
85     applies,
86     so the local 'point it at any of my videos' workflow is preserved. Setting an
87     allowed_root (a future hosted/tenant deployment) enforces containment.
88     """

```

```
87         if "\\x00" in path:
88             raise ValueError("video_path must not contain a null byte")
89         if allowed_root:
90             resolve_under_root(path, allowed_root)
91
```

#### **8. assistant (2026-07-02T04:58:31.343Z)**

You've hit your session limit · resets 1:10pm (Asia/Calcutta)